



DC – AI – Routing – Switching – Security – Wireless – Automation

## **PTX FIB Compression**

An HPE Juniper Networking Techpost

**Nicolas Fevrier**

2022-09-19

**HPE**



## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>PTX and ACX7000 FIB Compression</b> | <b>3</b> |
| 1.1      | TL;DR . . . . .                        | 3        |
| 1.2      | Introduction . . . . .                 | 3        |
| 1.3      | How Does it Work? . . . . .            | 5        |
| 1.3.1    | Shadowing . . . . .                    | 5        |
| 1.3.2    | Compression . . . . .                  | 5        |
| 1.3.3    | More Specific Prefixes . . . . .       | 6        |
| 1.4      | Support and Limitations . . . . .      | 7        |
| 1.5      | Implementation . . . . .               | 8        |
| 1.5.1    | Compression Test 1 . . . . .           | 9        |
| 1.5.2    | Compression Test 2 . . . . .           | 10       |
| 1.5.3    | Compression Test 3 . . . . .           | 14       |
| 1.6      | Does it Work in Production? . . . . .  | 17       |
| 1.7      | Best Case Scenario . . . . .           | 20       |
| 1.8      | What About the Churn? . . . . .        | 21       |
| 1.9      | Useful links . . . . .                 | 25       |
| 1.10     | Glossary . . . . .                     | 26       |
| 1.11     | Acknowledgements . . . . .             | 26       |

## List of Figures

|    |                                                                     |    |
|----|---------------------------------------------------------------------|----|
| 1  | Simple Shadowing Example . . . . .                                  | 5  |
| 2  | Simple Compression Example . . . . .                                | 5  |
| 3  | Example with Different Next-Hop Addresses . . . . .                 | 6  |
| 4  | More Specific Prefix Scenario. . . . .                              | 7  |
| 5  | Implementation of the Compression Algorithm in PTX Router . . . . . | 8  |
| 6  | Lab Topology . . . . .                                              | 9  |
| 7  | Radix Tree with Ideal Compression . . . . .                         | 9  |
| 8  | Same Topology but Different Advertisement . . . . .                 | 11 |
| 9  | New Radix Tree after new NH injection . . . . .                     | 12 |
| 10 | Advertisement of More Specific Prefixes . . . . .                   | 14 |
| 11 | Advertisement of More Two Specific Prefixes . . . . .               | 15 |
| 12 | Best Case Test Topology . . . . .                                   | 20 |
| 13 | Advertisement of 1M Contiguous Prefixes . . . . .                   | 21 |
| 14 | Additional Advertisement of Two /32 Prefixes . . . . .              | 22 |
| 15 | Churn Test – State 1 . . . . .                                      | 23 |
| 16 | Churn Test – State 2 . . . . .                                      | 24 |
| 17 | IXIA Results . . . . .                                              | 25 |

# 1 PTX and ACX7000 FIB Compression

**Nicolas Fevrier - 09/19/2022**

Did you know the internet table can be compressed significantly? We explain how the PTX and ACX7000 routers running Junos EVO are currently implementing FIB compression.

## 1.1 TL;DR

FIB Compression has been discussed for a very long time in this industry, but what most people probably don't know: it's already very efficiently deployed in many production networks. The maximum compression ratio we can get on today's internet table is 82% but we found in production networks it was reducing the FIB table by 50-65% in most cases.

This article has been initially written for PTX Series but the exact same feature is implemented on all ACX7000 routers (with the exception of the ACX7024).

## 1.2 Introduction

RIPE NCC announced in November 2019 they made their final IPv4 allocation, and still, the public internet table is growing at a constant rate, exceeding 930,000 entries at the time of this publication (September 2022). IPv4 table is a very constrained resource, you can easily imagine that IPv6 table is growing significantly faster and the entries are larger by nature. The [CIDR Report presents in their aggregation report](#) how different Autonomous Systems disaggregate their allocated prefixes.

## Aggregation Summary

The algorithm used in this report proposes aggregation only when there is a precise match using AS path so as to preserve traffic transit policy space ('holes').

--- 06Sep22 ---

ASnum NetsNow NetsAggr NetGain % Gain Description

|          |        |        |        |       |                                                                      |
|----------|--------|--------|--------|-------|----------------------------------------------------------------------|
| Table    | 930631 | 515865 | 414766 | 44.6% | All ASes                                                             |
| AS8151   | 10886  | 1761   | 9125   | 83.8% | Uninet S.A. de C.V., MX                                              |
| AS9808   | 9078   | 1901   | 7177   | 79.1% | CHINAMOBILE-CN China Mobile Communications Group Co., Ltd., CN       |
| AS12479  | 6912   | 336    | 6576   | 95.1% | UNI2-AS, ES                                                          |
| AS7545   | 5922   | 1430   | 4492   | 75.9% | TPG-INTERNET-AP TPG Telecom Limited, AU                              |
| AS39891  | 4010   | 128    | 3882   | 96.8% | ALJAWWALSTC-AS, SA                                                   |
| AS8551   | 3992   | 210    | 3782   | 94.7% | BEZEQ-INTERNATIONAL-AS Bezeqint Internet Backbone, IL                |
| AS7155   | 3898   | 159    | 3739   | 95.9% | VIASAT-SP-BACKBONE, US                                               |
| AS7713   | 3518   | 529    | 2989   | 85.0% | TELKOMNET-AS-AP PT Telekomunikasi Indonesia, ID                      |
| AS6327   | 3456   | 478    | 2978   | 86.2% | SHAW, CA                                                             |
| AS9121   | 6861   | 4099   | 2762   | 40.3% | TTNET, TR                                                            |
| AS22773  | 3868   | 1137   | 2731   | 70.6% | ASN-CXA-ALL-CCI-22773-RDC, US                                        |
| AS4538   | 4944   | 2286   | 2658   | 53.8% | ERX-CERNET-BKB China Education and Research Network Center, CN       |
| AS16509  | 6981   | 4403   | 2578   | 36.9% | AMAZON-02, US                                                        |
| AS7552   | 3582   | 1108   | 2474   | 69.1% | VIETEL-AS-AP Viettel Group, VN                                       |
| AS45899  | 3461   | 1201   | 2260   | 65.3% | VNPT-AS-VN VNPT Corp, VN                                             |
| AS9498   | 2427   | 260    | 2167   | 89.3% | BBIL-AP BHARTI Airtel Ltd., IN                                       |
| AS396982 | 2533   | 390    | 2143   | 84.6% | GOOGLE-CLOUD-PLATFORM, US                                            |
| AS11492  | 4575   | 2450   | 2125   | 46.4% | CABLEONE, US                                                         |
| AS45090  | 2180   | 411    | 1769   | 81.1% | TENCENT-NET-AP Shenzhen Tencent Computer Systems Company Limited, CN |
| AS12389  | 2934   | 1170   | 1764   | 60.1% | ROSTELECOM-AS, RU                                                    |
| AS24560  | 2193   | 512    | 1681   | 76.7% | AIRTELBROADBAND-AS-AP Bharti Airtel Ltd., Telemedia Services, IN     |
| AS4766   | 2477   | 810    | 1667   | 67.3% | KIXS-AS-KR Korea Telecom, KR                                         |
| AS4755   | 2123   | 555    | 1568   | 73.9% | TATACOMM-AS TATA Communications formerly VSNL is Leading ISP, IN     |
| AS18403  | 1888   | 359    | 1529   | 81.0% | FPT-AS-AP FPT Telecom Company, VN                                    |
| AS209    | 2358   | 954    | 1404   | 59.5% | CENTURYLINK-US-LEGACY-QWEST, US                                      |
| AS6849   | 2788   | 1384   | 1404   | 50.4% | UKRTELNET, UA                                                        |
| AS45609  | 1825   | 504    | 1321   | 72.4% | BHARTI-MOBILITY-AS-AP Bharti Airtel Ltd. AS for GPRS Service, IN     |
| AS56046  | 1379   | 126    | 1253   | 90.9% | CMNET-JIANGSU-AP China Mobile communications corporation, CN         |
| AS56047  | 1594   | 357    | 1237   | 77.6% | CMNET-HUNAN-AP China Mobile communications corporation, CN           |
| AS20115  | 2249   | 1056   | 1193   | 53.0% | CHARTER-20115, US                                                    |
| Total    | 116892 | 32464  | 84428  | 72.2% | Top 30 total                                                         |

Programmed in FIB

Not Programmed in FIB

Received Prefix

Aggregated Prefix

- In light blue, we represent the routes received from the BGP neighbors. Note that routes can be learned from any source, BGP, IGP, local, or static, ... It's not relevant in this context, we just happened to use BGP to easily advertise large tables.
- In green, we represent prefixes compressed by the algorithm.
- In dotted line, the prefixes are not installed in hardware.
- In solid line, the prefixes are pushed in FIB.
- High-level description of the mechanisms used to compress the FIB
- Implementation on PTX devices: support and limitations
- Verification of these principles with concrete examples in the lab
- How far we can compress today's internet table? What could be the best case?
- The demonstration we don't lose a single packet when reshuffling the compression tree
- How efficiently it compresses routes in our customer's networks?

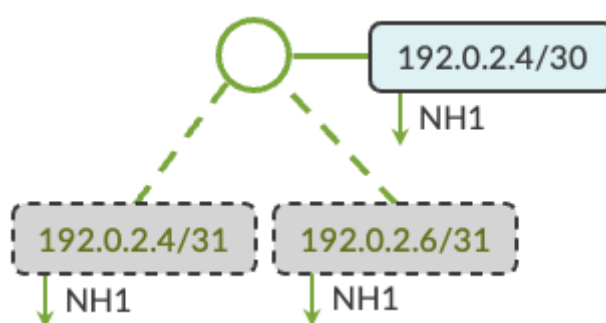
### 1.3 How Does it Work?

The mechanism is following two simple rules:

- Shadowing: if a superset prefix is already present in the FIB table, don't install more specific routes with the same Next-Hop (NH) address
- Compression: if several contiguous prefixes with the same NH can be "summarized" to a superset prefix, just push this aggregate.

Several exceptions / configuration may prevent a prefix to be compressed, we will list them in the implementation section.

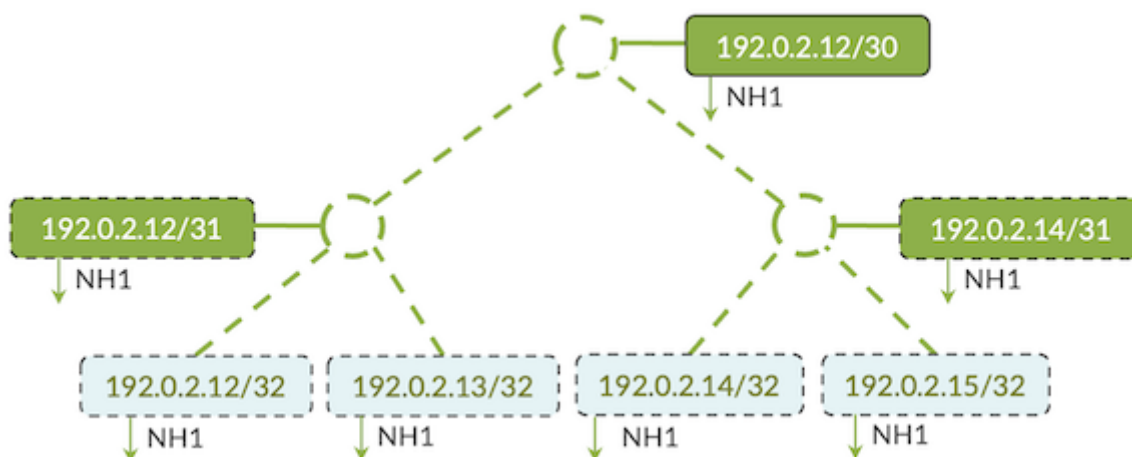
#### 1.3.1 Shadowing



**Figure 1:** Simple Shadowing Example

In the Diagram 2 example, three prefixes are received from a BGP speaker, all pointing to the same NH1. The two /31s are "covered" by the superset /30. 192.0.2.4/31 and 192.0.2.6/31 are "shadowed" and not pushed in the PFE FIB, we will only install the /30.

#### 1.3.2 Compression



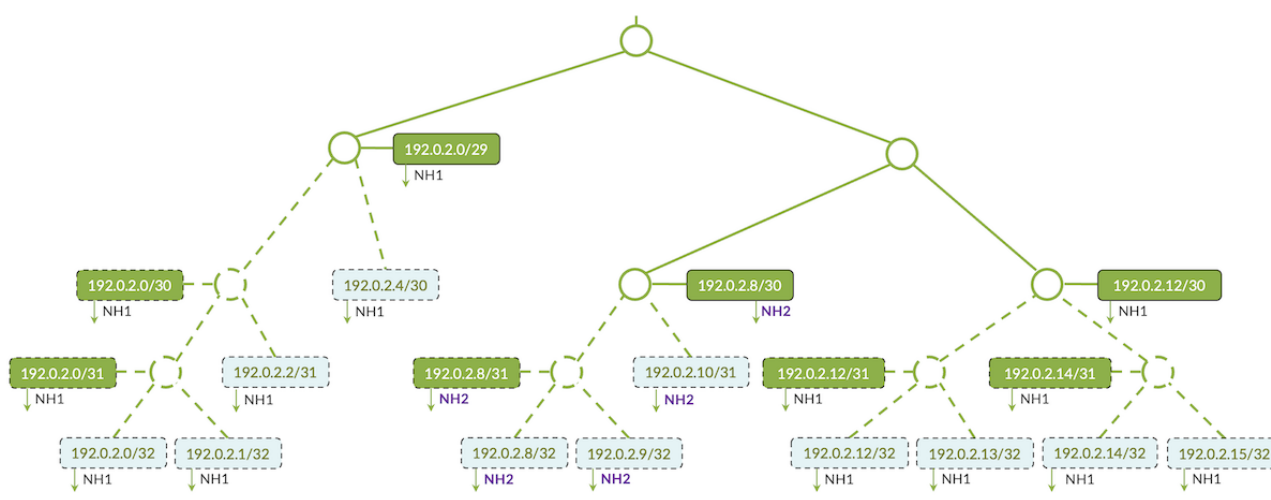
**Figure 2:** Simple Compression Example

In Diagram 3, we demonstrate two compression levels:

- .12/32 and .13/32 can be aggregated to .12/31
- .13/32 and .14/32 can be aggregated to .14/31
- And these two aggregates can be summarized themselves to .12/30

We are installing a single route, 192.0.2.12/30.

Keep in mind these prefixes need to have the same forwarding behavior. That means, the same next-hop address:



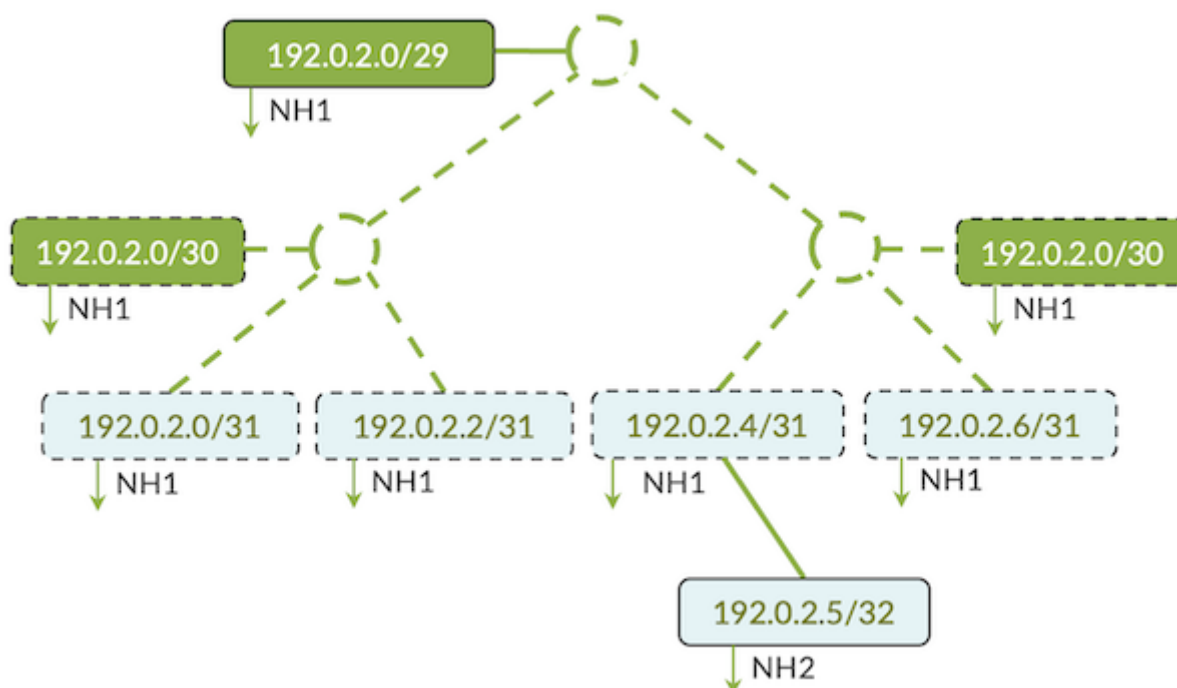
**Figure 3:** Example with Different Next-Hop Addresses

The example in Diagram 4 illustrates why it's not possible to aggregate these 12 prefixes to a unique 192.0.2.0/28: three of them “in the middle” don't have the same Next-Hop address. Yet, we can summarize this tree into three routes.

Note: the aggregation is not limited to prefixes of similar length. We gathered multiple /32s and multiple /31s prefixes to generated 192.0.2.0/29.

### 1.3.3 More Specific Prefixes

The following example is showing another level of subtleties for the compression algorithm.



**Figure 4:** More Specific Prefix Scenario.

In this situation, the router received 5 prefixes:

- Four of them can be aggregated to 192.0.2.0/29
- The last one 192.0.2.5/32 is more specific than the received 192.0.2.4/31 but is pointing to a different NH2, so it's not "shadowed".

This 5th prefix is not breaking the tree structure and doesn't affect the compression. We will install two prefixes in hardware: 192.0.2.0/29→NH1 and 192.0.2.5/32→NH2

## 1.4 Support and Limitations

The compression ratio will be different from customer to customer, and even between two routers in different places/roles in the network. Not only the number but the variety of next-hop addresses will influence the algorithm performance.

Later in this article, we will demonstrate how far we can compress the public view using a single next-hop (the best possible case). And we will present the FIB space reduction, measured in different live networks.

The compression algorithm handles unicast IPv4 and IPv6 prefixes. The advertising protocols (or even local, static, ...) used to learn these routes are not important because compression is performed at the FIB level. It works for routes in inet.0 or L3VPN VRFs. Finally, it has no impact on uRPF check.

FIB compression is not implemented for multicast routes. The size of the multicast tables wouldn't justify it.

Some "features" can prevent routes from being aggregated, like:

- [SCU/DCU accounting \(source/destination class usage\)](#)



In such cases, the specific routes will not be compressed.

Today, the first routers to natively support the features are the PTX powered by Express 4 chipset and running Junos EVO:

- PTX10001-36MR
- LC1201 and LC1202 line cards in PTX10000 chassis

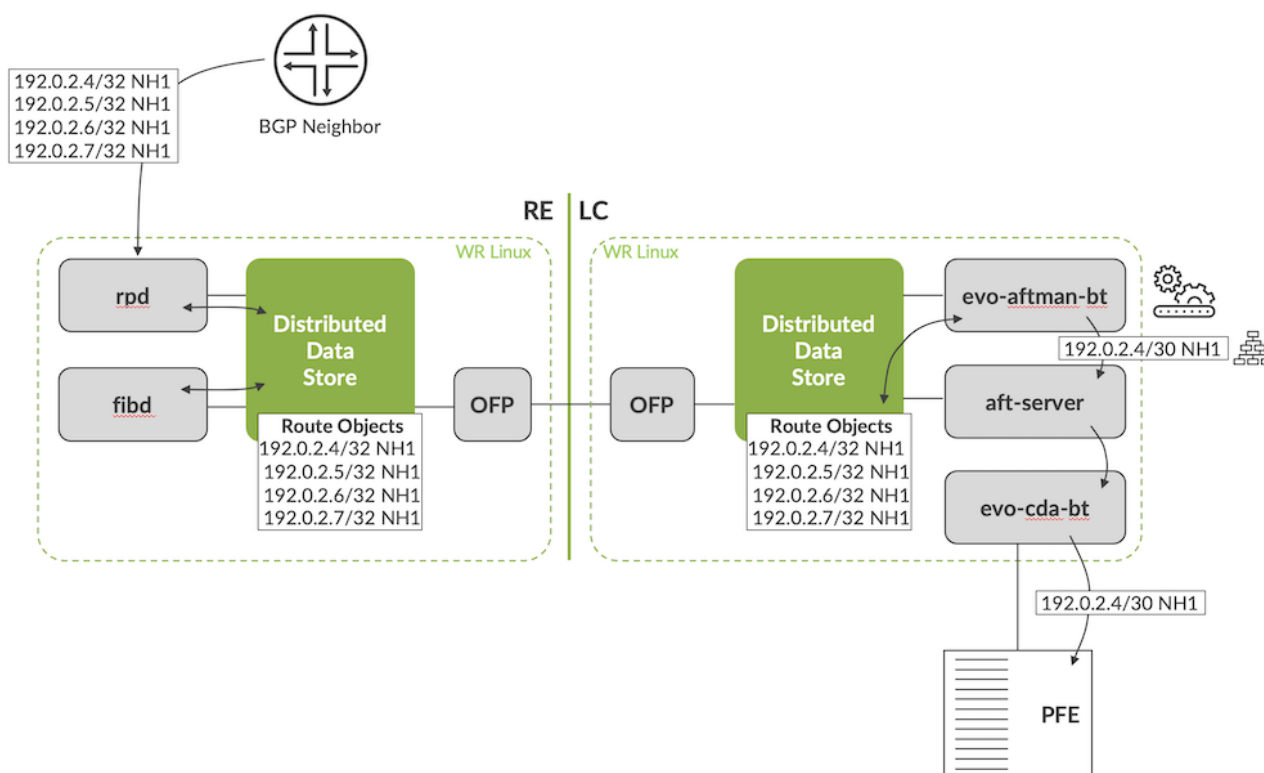
Other platforms based on Junos EVO will implement the same algorithm soon.

FIB compression has been introduced for the PTX platforms listed above starting from 21.2R1. The feature is enabled by default, it doesn't require any specific configuration.

## 1.5 Implementation

The algorithm is implemented at the line card CPU by the “evo-aftman-bt” process. You notice it doesn't happen at the Routing Engine (RE) level but in a distributed fashion, as close as possible to the PFE.

The routes are not modified in the RIB, or other protocol tables, therefore, compression does not affect redistribution.



**Figure 5:** Implementation of the Compression Algorithm in PTX Router

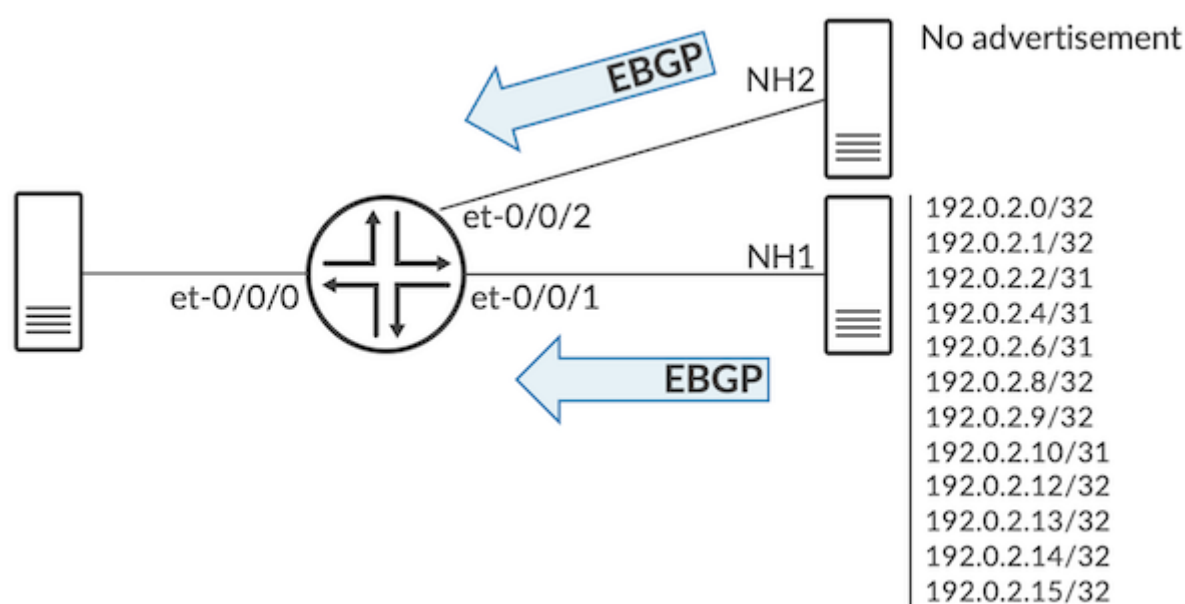
Diagram 6 represents a chassis with Express 4 Line Cards.

In a fixed form-factor router like PTX10001-36MR, it's simplified: we don't need to replicate the route objects in the Distributed DataStore (DDS) for example. In our chassis example, the prefixes are distributed via this datastore and the evo-aftman-bt will construct the radix tree. The compression happens here. Eventually, the evo-cda-bt process will program the compressed FIB in the PFE hardware table.

Let's have a look at the behavior of this algorithm in the lab with concrete examples. We will use a router connected to a route and traffic generator, symbolized with this icon in the following diagrams:

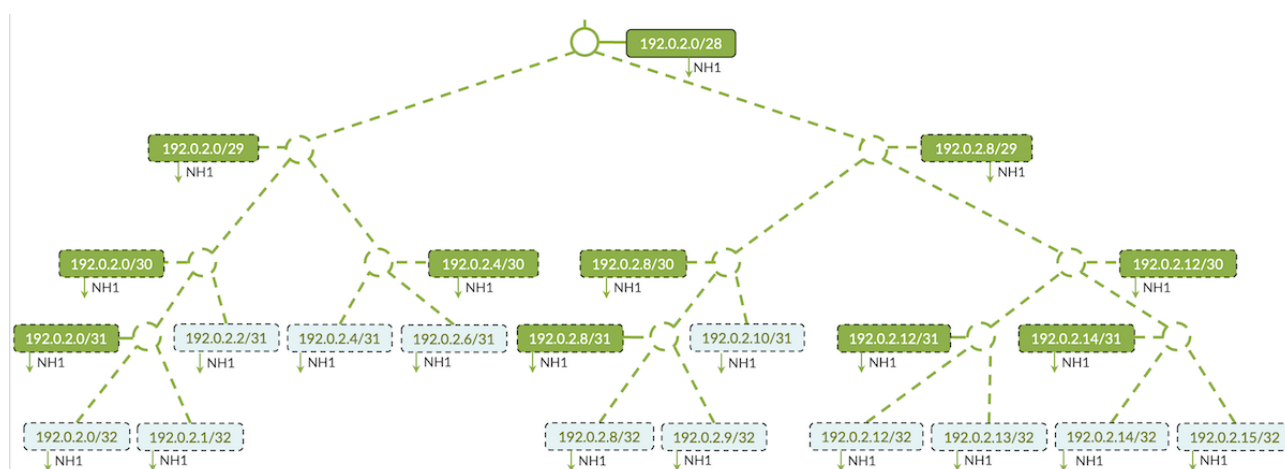


### 1.5.1 Compression Test 1



**Figure 6:** Lab Topology

These /32 and /31 routes are pointing to the same NH address and are “contiguous”, they can be aggregated to 192.0.2.0/28



**Figure 7:** Radix Tree with Ideal Compression

We take a look at the aggregated routes:

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select aggregate
2
3 Index Destination NH Id NH Type NH Token GUID
4 -----
5 0 192.0.2.0/28 13027 software 6068 0
6 0 192.0.2.0/29 13027 software 6068 0
7 0 192.0.2.0/30 13027 software 6068 0
8 0 192.0.2.0/31 13027 software 6068 0
9 0 192.0.2.4/30 13027 software 6068 0
10 0 192.0.2.8/29 13027 software 6068 0
11 0 192.0.2.8/30 13027 software 6068 0
12 0 192.0.2.8/31 13027 software 6068 0
13 0 192.0.2.12/30 13027 software 6068 0
14 0 192.0.2.12/31 13027 software 6068 0
15 0 192.0.2.14/31 13027 software 6068 0
16
17 regress@rtme-ptx10:pfe>

```

This output shows the “recursive” compression:

- The two last lines:

192.0.2.12/32 and 192.0.2.13/32 are compressed to 192.0.2.12/31 (in blue) 192.0.2.14/32 and 192.0.2.15/32 are compressed to 192.0.2.14/31 (in blue) - both /31 aggregates from the previous step are also aggregated into 192.0.2.2/30 (in green) - And it continues level after level up to 192.0.2.0/28 - The NH type “software” represents the entries created by the compression algorithm.

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.14/32 detail
2 Protocol : IPv4
3 Table : default
4 Prefix : 192.0.2.14 (primary)
5 NH : 13027 (software)
6 Flags : 0x00008000
7 Details :
8   guid : 833230259553
9   type : user
10  nhid : 13027
11  Forwarding state:
12    installed? : no
13    (Installed parent: 192.0.2.0/28)
14
15 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.0/28 detail
16 Protocol : IPv4
17 Table : default
18 Prefix : 192.0.2.0/28 (primary)
19 NH : 13027 (software)
20 Flags : 0x00008000
21 Details :
22   guid : 0
23   type : user
24   nhid : 13027
25   Forwarding state:
26     installed? : yes
27     nh-token : 6068
28
29 regress@rtme-ptx10:pfe>

```

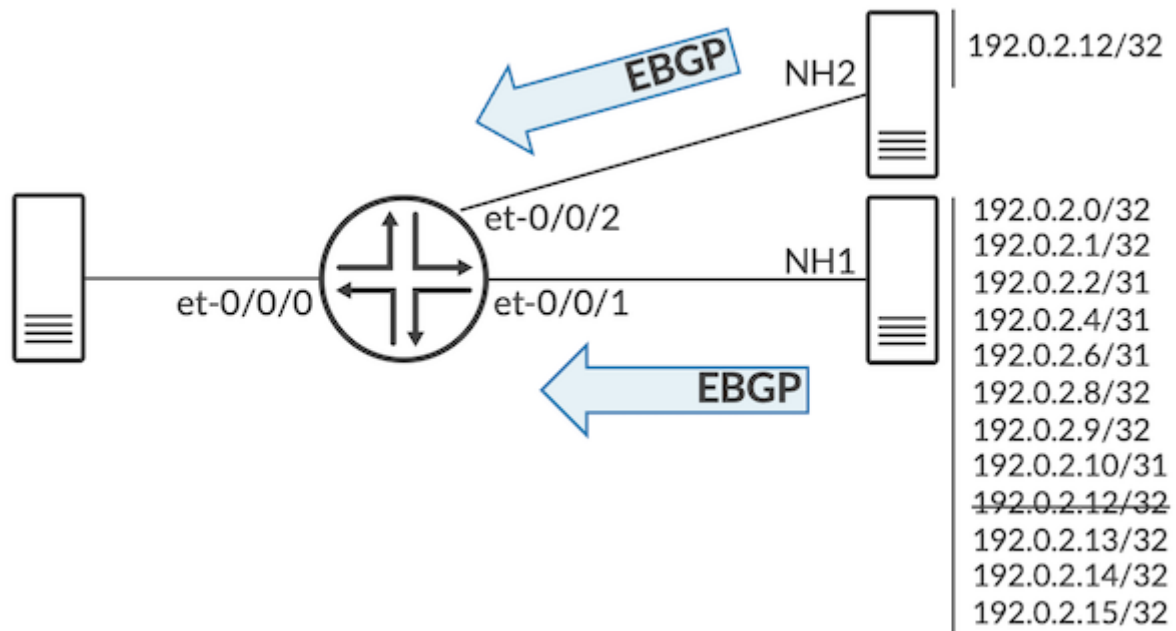
In this last output, we check the handling of a specific prefix (192.0.2.14/32) and we can notice it’s not installed in favor of the parent prefix 192.0.2.0/28.

### 1.5.2 Compression Test 2

Now in the second example:

- We start from the test 1 conditions (twelve contiguous prefixes aggregated into a /28)
- We stop advertising 192.0.2.12/32 from NH1, and announced it from a new peer, with a new next hop address NH2.

- The twelve routes can no longer be aggregated in one, It changes the structure of the tree and impacts the compression.



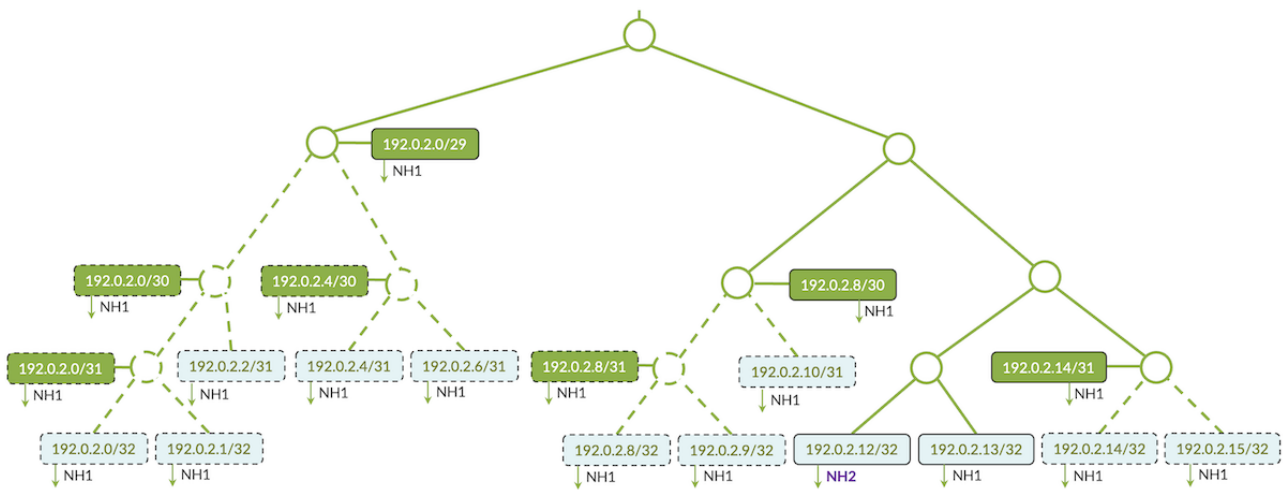
**Figure 8:** Same Topology but Different Advertisement

```

1 regress@rtme-ptx10> show bgp summary
2
3 Threading mode: BGP I/O
4 Default eBGP mode: advertise - accept, receive - accept
5 Groups: 4 Peers: 6 Down peers: 4
6 Table Tot Paths Act Paths Suppressed Histry Damp State Pending
7 inet.0
8          12 12 0 0 0 0
9 inet6.0
10         0 0 0 0 0 0
11 Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State|#Active/Received/Accepted/Damped...
12 15.1.1.2 65001 500 515 0 9 3:51:24 Establ
13   inet.0: 11/11/11/0
14 15.1.2.2 65002 4 3 0 5 2 Establ
15   inet.0: 1/1/1/0
16 15.1.3.2 65003 0 0 0 5 4:22:05 Active
17 2002:15:1:1::2 65001 0 0 0 2 4:38:27 Active
18 2002:15:1:2::2 65002 0 0 0 2 4:39:15 Active
19 2002:15:1:3::2 65003 0 0 0 3 4:22:04 Active
20
21 regress@rtme-ptx10>

```

The neighbour 15.1.2.2 advertises a single route 192.0.2.12/32 modifying the structure of the tree, leading to a less efficient compression ratio:



**Figure 9:** New Radix Tree after new NH injection

We verify the aggregation and the prefixes not installed in FIB with the following CLI.

The aggregated routes are those computed by the algorithm and represented in green in the diagram. The uninstalled routes are represented in dotted lines in the diagram.

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select aggregate
2
3 Index Destination NH Id NH Type NH Token GUID
4 -----
5 0 192.0.2.0/29 13027 software 6068 0
6 0 192.0.2.0/30 13027 software 6068 0
7 0 192.0.2.0/31 13027 software 6068 0
8 0 192.0.2.4/30 13027 software 6068 0
9 0 192.0.2.8/30 13027 software 6068 0
10 0 192.0.2.8/31 13027 software 6068 0
11 0 192.0.2.14/31 13027 software 6068 0
12
13 regress@rtme-ptx10:pfe> show route proto ip index 0 select uninstalled
14
15 Index Destination NH Id NH Type NH Token GUID
16 -----
17 0 192.0.2.0/30 13027 software 6068 0
18 0 192.0.2.0/31 13027 software 6068 0
19 0 192.0.2.0 13027 software 6068 833230259556
20 0 192.0.2.1 13027 software 6068 833230259555
21 0 192.0.2.2/31 13027 software 6068 833230259554
22 0 192.0.2.4/30 13027 software 6068 0
23 0 192.0.2.4/31 13027 software 6068 833230259549
24 0 192.0.2.6/31 13027 software 6068 833230259558
25 0 192.0.2.8/31 13027 software 6068 0
26 0 192.0.2.8 13027 software 6068 833230259559
27 0 192.0.2.9 13027 software 6068 833230259552
28 0 192.0.2.10/31 13027 software 6068 833230259548
29 0 192.0.2.14 13027 software 6068 833230259553
30 0 192.0.2.15 13027 software 6068 833230259551
31
32 regress@rtme-ptx10:pfe>

```

If you don't want to verify each prefix one by one, count them: we have seven aggregate entries in the output (and seven green boxes in the diagram). In the same manner, we have fourteen entries in the uninstalled CLI output (and fourteen dotted line boxes in the diagram too).

We can also check specific prefixes and see which ones are installed or not. If not, the output gives us the installed parent.

```

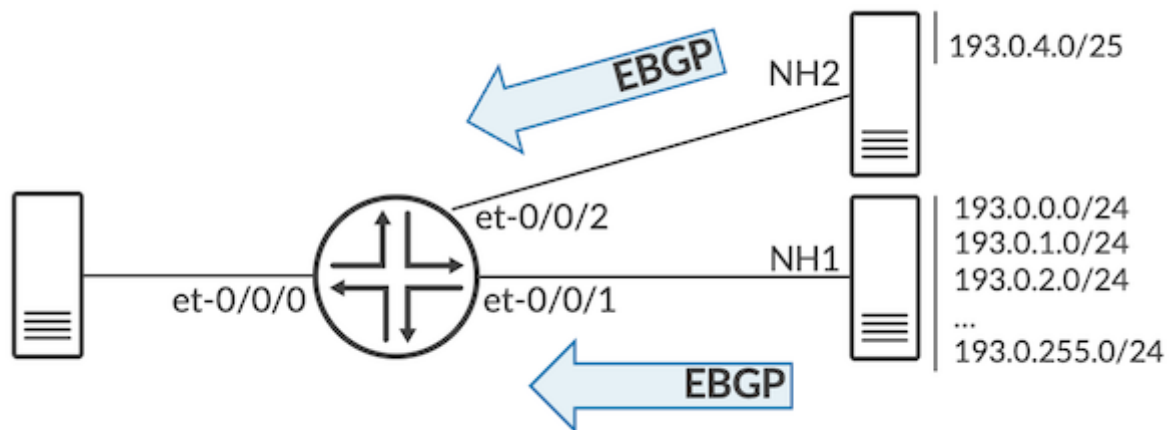
1 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.1/32 detail
2 Protocol : IPv4
3 Table : default
4 Prefix : 192.0.2.1 (primary)

```

```
5  NH : 13027 (software)
6  Flags : 0x00008000
7  Details :
8      guid : 833230259555
9      type : user
10     nhid : 13027
11     Forwarding state:
12         installed? : no <<< Not Installed
13     (Installed parent: 192.0.2.0/29) <<< Parent prefix
14
15 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.0/29 detail
16 Protocol : IPv4
17 Table : default
18 Prefix : 192.0.2.0/29 (primary)
19 NH : 13027 (software)
20 Flags : 0x00008000
21 Details :
22     guid : 0
23     type : user
24     nhid : 13027
25     Forwarding state:
26         installed? : yes
27     nh-token : 6068
28
29 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.12/32 detail
30 Protocol : IPv4
31 Table : default
32 Prefix : 192.0.2.12 (primary)
33 NH : 13026 (software)
34 Flags : 0x00008000
35 Details :
36     guid : 833230259572
37     type : user
38     nhid : 13026
39     Forwarding state:
40         installed? : yes <<< Installed
41     nh-token : 607
42
43 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.13/32 detail
44 Protocol : IPv4
45 Table : default
46 Prefix : 192.0.2.13 (primary)
47 NH : 13027 (software)
48 Flags : 0x00008000
49 Details :
50     guid : 833230259550
51     type : user
52     nhid : 13027
53     Forwarding state:
54         installed? : yes
55     nh-token : 6068
56
57 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.14/32 detail
58 Protocol : IPv4
59 Table : default
60 Prefix : 192.0.2.14 (primary)
61 NH : 13027 (software)
62 Flags : 0x00008000
63 Details :
64     guid : 833230259553
65     type : user
66     nhid : 13027
67     Forwarding state:
68         installed? : no
69     (Installed parent: 192.0.2.14/31)
70
71 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 192.0.2.14/31 detail
72 Protocol : IPv4
73 Table : default
74 Prefix : 192.0.2.14/31 (primary)
75 NH : 13027 (software)
76 Flags : 0x00008000
77 Details :
78     guid : 0
79     type : user
80     nhid : 13027
81     Forwarding state:
82         installed? : yes
83     nh-token : 6068
84
85 regress@rtme-ptx10:pfe>
```

### 1.5.3 Compression Test 3

This other test will illustrate the “more specific prefix” principle detailed earlier. A large block of contiguous /24s is aggregated and a more specific /25 with a different next-hop is added in the mix:



**Figure 10:** Advertisement of More Specific Prefixes

We look at the installed prefixes:

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select installed
2
3
4 Index Destination NH Id NH Type NH Token GUID
5 -----
6 0 default 34 discard 1140 833223655665
7 0 0.0.0.0 34 discard 1140 622770257990
8 0 12.1.1.1 11002 local 1308 841813590953
9 0 15.1.1/24 11009 resolve 1730 841813591064
10 <SNIP>
11 0 15.1.3.2 53063 unicast 6035 721554517737
12 0 15.1.3.255 11034 bcast 5464 841813591844
13 0 193.0/16 13032 software 6094 0
14 0 193.0.4.0/25 13034 software 6098 833230392891
15 0 224/4 35 mdiscard 1141 622770257992
16 0 224.0.0.1 31 mcast 1137 622770257985
17 0 255.255.255.255 32 bcast 1138 622770257987
18
19 regress@rtme-ptx10:pfe>

```

The presence of the 193.0.4.0/25 didn’t “break” the tree structure. We have programmed the aggregate 193.0.0.0/16→NH1 and 193.0.4.0/25→NH2.

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.4.0/25 detail
2 Protocol : IPv4
3 Table : default
4 Prefix : 193.0.4.0/25 (primary)
5 NH : 13034 (software)
6 Flags : 0x00008000
7 Details :
8   guid : 833230392891
9   type : user
10  nhid : 13034
11  Forwarding state:
12    installed? : yes
13    nh-token : 6098
14
15 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.5.0/24 detail

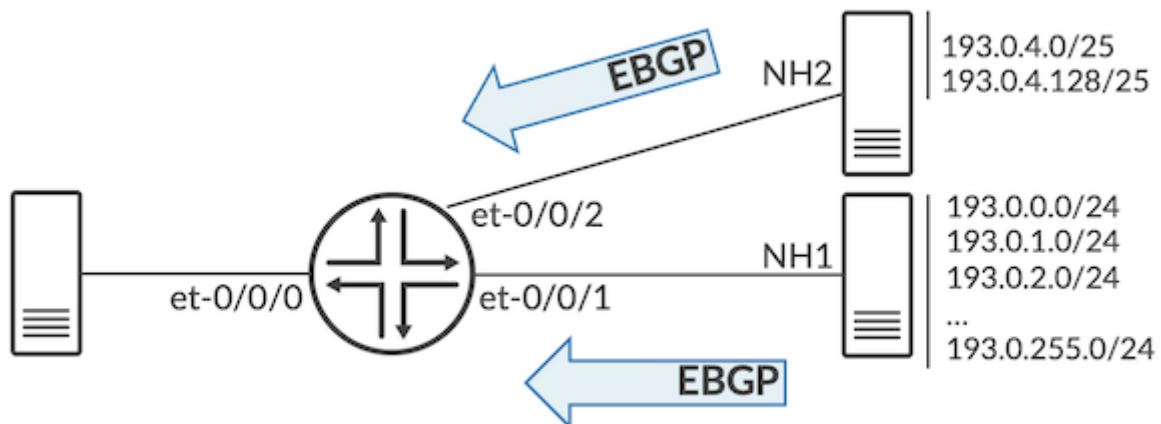
```

```

16 Protocol : IPv4
17 Table : default
18 Prefix : 193.0.5/24 (primary)
19 NH : 13032 (software)
20 Flags : 0x00008000
21 Details :
22     guid : 833230392633
23     type : user
24     nhid : 13032
25 Forwarding state:
26     installed? : no
27     (Installed parent: 193.0/16)
28
29 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.0.0/16 detail
30 Protocol : IPv4
31 Table : default
32 Prefix : 193.0/16 (primary)
33 NH : 13032 (software)
34 Flags : 0x00008000
35 Details :
36     guid : 0
37     type : user
38     nhid : 13032
39 Forwarding state:
40     installed? : yes
41     nh-token : 6094
42
43
44 regress@rtme-ptx10:pfe>

```

Now, just for the “fun” of the experience (don’t judge me), let’s see what happens if we advertise two contiguous /25 prefixes instead of just one:



**Figure 11:** Advertisement of More Two Specific Prefixes

Let’s take a look at the prefixes installed in hardware:

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select installed
2
3 Index Destination NH Id NH Type NH Token GUID
4 -----
5 0 default 34 discard 1140 833223655665
6 0 0.0.0.0 34 discard 1140 622770257990
7 <SNIP>
8 0 15.1.3.1 11036 local 5473 841813591848
9 0 15.1.3.2 53063 unicast 6035 721554517737
10 0 15.1.3.255 11034 bcast 5464 841813591844
11 0 193.0.0/22 13032 software 6094 0
12 0 193.0.4/24 13034 software 6098 0
13 0 193.0.5/24 13032 software 6094 833230392633
14 0 193.0.6/23 13032 software 6094 0
15 0 193.0.8/21 13032 software 6094 0
16 0 193.0.16/20 13032 software 6094 0
17 0 193.0.32/19 13032 software 6094 0

```



```

18 0 193.0.64/18 13032 software 6094 0
19 0 193.0.128/17 13032 software 6094 0
20 0 224/4 35 mdiscard 1141 622770257992
21 0 224.0.0.1 31 mcast 1137 622770257985
22 0 255.255.255.255 32 bcast 1138 622770257987
23
24 regress@rtme-ptx10:pfe>

```

Interestingly, the compression to 193.0.0.0/16 has been “broken” into multiple more specific aggregates (from /17 to /23)

It’s an expected behavior considering 193.0.4.0/25→NH2 and 193.0.4.128/25→NH2 have been aggregated into a 193.0.4.0/24→NH2

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.4.0/25 detail
2 Protocol : IPv4
3
4 Table : default
5 Prefix : 193.0.4.0/25 (primary)
6 NH : 13034 (software)
7 Flags : 0x00008000
8 Details :
9     guid : 833230392891
10    type : user
11    nhid : 13034
12    Forwarding state:
13    installed? : no
14    (Installed parent: 193.0.4/24)
15
16 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.4.0/24 detail
17 Protocol : IPv4
18 Table : default
19 Prefix : 193.0.4/24 (primary)
20 NH : 13034 (software)
21 Flags : 0x00008000
22 Details :
23     guid : 0
24     type : user
25     nhid : 13034
26     Forwarding state:
27     installed? : yes
28     nh-token : 6098
29
30 regress@rtme-ptx10:pfe>

```

It does replace the original 193.0.4.0/24→NH1, therefore the system can’t summarize to 193.0/16→NH1 anymore.

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.2.0/24 detail
2
3 Protocol : IPv4
4 Table : default
5 Prefix : 193.0.2/24 (primary)
6 NH : 13032 (software)
7 Flags : 0x00008000
8 Details :
9     guid : 833230392630
10    type : user
11    nhid : 13032
12    Forwarding state:
13    installed? : no
14    (Installed parent: 193.0.0/22)
15
16 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.0.0/22 detail
17 Protocol : IPv4
18 Table : default
19 Prefix : 193.0.0/22 (primary)
20 NH : 13032 (software)
21 Flags : 0x00008000
22 Details :
23     guid : 0
24     type : user
25     nhid : 13032
26     Forwarding state:
27     installed? : yes
28     nh-token : 6094

```

```

29 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.128.0/24 detail
30 Protocol : IPv4
31 Table : default
32 Prefix : 193.0.128/24 (primary)
33 NH : 13032 (software)
34 Flags : 0x00008000
35 Details :
36     guid : 833230392756
37     type : user
38     nhid : 13032
39 Forwarding state:
40     installed? : no
41     (Installed parent: 193.0.128/17)
42
43 regress@rtme-ptx10:pfe> show route proto ip index 0 prefix 193.0.128.0/17 detail
44 Protocol : IPv4
45 Table : default
46 Prefix : 193.0.128/17 (primary)
47 NH : 13032 (software)
48 Flags : 0x00008000
49 Details :
50     guid : 0
51     type : user
52     nhid : 13032
53 Forwarding state:
54     installed? : yes
55     nh-token : 6094
56
57 regress@rtme-ptx10:pfe>

```

With these examples, the reader should clearly understand the algorithm's behavior.

## 1.6 Does it Work in Production?

As mentioned earlier, this feature is activated by default on the Express 4 routers since Junos release 21.2R1: it has been deployed in many production networks and we can verify the compression performance in real conditions.

We collected and anonymized data:

```

1 user@ptx36mr:pfe> show route summary
2
3 IPv4 Route Tables:
4 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
5 -----
6 Default 913170 131812824 895870 354261 368514 59
7 1 0 0 0 0 0 0
8 51 5 520 5 0 5 0
9 52 221102 31465408 217778 81450 93102 57
10 53 12 1248 11 0 11 0
11 36738 9 936 9 0 9 0
12
13 MPLS Route Tables:
14 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
15 -----
16 Default 522 54288 522 0 522 -
17 54 1 104 1 0 1 -
18
19 IPv6 Route Tables:
20 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
21 -----
22 Default 155979 22288448 154262 58333 59732 62
23 1 0 0 0 0 0 0
24 51 6 624 6 0 6 0
25 52 29127 4226144 29007 11509 10992 63
26 53 7 728 7 0 7 0
27 36738 14 1456 14 0 14 0
28
29 CLNP Route Tables:
30 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
31 -----
32 Default 2 208 2 0 2 -
33 51 1 104 1 0 1 -

```

```

34 52 1 104 1 0 1 -
35 53 1 104 1 0 1 -
36
37 user@ptx36mr:pfe> show route compression
38
39 Index Proto Prefixes Aggregate Installed Comp(%)
40 -----
41 0 IPv4 895872 354255 368519 59
42 1 IPv4 0 0 0 0
43 51 IPv4 5 0 5 0
44 52 IPv4 217778 81446 93103 57
45 53 IPv4 11 0 11 0
46 36738 IPv4 9 0 9 0
47 0 IPv6 154259 58332 59730 62
48 1 IPv6 0 0 0 0
49 51 IPv6 6 0 6 0
50 52 IPv6 29006 11509 10991 63
51 53 IPv6 7 0 7 0
52 36738 IPv6 14 0 14 0
53
54 user@ptx36mr:pfe> show nh summary
55 Type Count Max Count
56 Discard 16 16
57 Reject 19 16
58 Unicast 848 871
59 Unilist 393 490
60 Indexed 0 0
61 Indirect 137 137
62 Hold 2 28
63 Resolve 78 78
64 XResolve 0 0
65 Local 94 94
66 Receive 114 114
67 multirt 0 0
68 Bcast 16 16
69 Mcast 12 12
70 Mgroup 0 0
71 MDiscard 12 12
72 Table 16 16
73 Deny 12 12
74 Composite 116 225
75 Software 906 907
76 Aggregate 1834 1837
77
78 Total number of NH = 4625
79
80 user@ptx36mr:pfe>

```

“Indirect” represents the next-hop addresses used by BGP in our case.

In this chart, we have RIB table, number of next=hop and the compression efficiency (representing the FIB space reduction).

|                 | RIB Table size | Number of NH | FIB Space Reduction |
|-----------------|----------------|--------------|---------------------|
| Customer A IPv4 | 913170         | 137          | 59%                 |
| Customer A IPv6 | 155979         | 137          | 62%                 |
| Customer B IPv4 | 884835         | 1600         | 55%                 |
| Customer B IPv6 | 149367         | 1600         | 60%                 |
| Customer C IPv4 | 968587         | 2030         | 56%                 |
| Customer C IPv6 | 153519         | 2030         | 60%                 |

When the feature has been introduced in 2020, we also measured the compression in diverse networks (IPv4 and IPv6 public tables were slightly smaller).

|                 | RIB Table size | Number of NH | FIB Space Reduction |
|-----------------|----------------|--------------|---------------------|
| Customer 1 IPv4 | 814621         | 133          | 69%                 |
| Customer 2 IPv4 | 816791         | 148          | 61%                 |
| Customer 3 IPv4 | 801872         | 1000         | 69%                 |
| Customer 4 IPv4 | 838589         | 59           | 86%                 |
| Customer 5 IPv4 | 958854         | 2538         | 55%                 |
| Customer 6 IPv4 | 967325         | 1815         | 61%                 |
| Customer 7 IPv4 | 811385         | 453          | 58%                 |
| Customer 8 IPv6 | 83313          | 21           | 54%                 |

The recent examples are showing a compression performance ranging from 50% to 62%. The marketing message “compression **doubles** the FIB space”, is even conservative in some cases.

Every network will show a different level of compression depending on the way routes are mapped to next hop addresses:

- In the best case, all best routes will point to a unique NH (that’s what we test in next section).
- In the worst pathological case, all contiguous routes are using different next-hop addresses and can not be compressed.

To illustrate the worst case, we are taking a portion of the potaroo routes used in next section. They have three BGP feeds / NH addresses, and the variety of NH prevents compression for this specific series of routes.

```

1 BGP table version is 0, local router ID is 203.133.248.2
2 Status codes: s suppressed, d damped, h history, * valid, > best, = multipath,
3               i internal, r RIB-failure, S Stale, R Removed
4 Origin codes: i - IGP, e - EGP, ? - incomplete
5
6   Network Next Hop Metric LocPrf Weight Path
7   * 1.0.132.0/24 202.12.28.1 0 4777 4713 2914 38040 23969 ?
8   * > 203.119.104.1 0 4608 4635 38040 23969 ?
9   * 203.119.104.2 0 4608 24115 38040 23969 ?
10  * 1.0.133.0/24 202.12.28.1 0 4777 4713 2914 38040 23969 ?
11  * 203.119.104.1 0 4608 4635 38040 23969 ?
12  * > 203.119.104.2 0 4608 24115 38040 23969 ?
13  * 1.0.136.0/24 202.12.28.1 0 4777 4713 2914 38040 23969 ?
14  * > 203.119.104.1 0 4608 4635 38040 23969 ?
15  * 203.119.104.2 0 4608 24115 38040 23969 ?
16  * > 1.0.137.0/24 202.12.28.1 0 4777 6939 4651 23969 i
17  * 203.119.104.1 0 4608 24115 6939 4651 23969 i
18  * 203.119.104.2 0 4608 24115 6939 4651 23969 i
19  * 1.0.138.0/24 202.12.28.1 0 4777 4713 2914 38040 23969 ?
20  * > 203.119.104.1 0 4608 4635 38040 23969 ?
21  * 203.119.104.2 0 4608 24115 38040 23969 ?
22  * 1.0.139.0/24 202.12.28.1 0 4777 4713 2914 38040 23969 ?
23  * 203.119.104.1 0 4608 4635 38040 23969 ?
24  * > 203.119.104.2 0 4608 24115 38040 23969 ?
25  * > 1.0.141.0/24 202.12.28.1 0 4777 6939 4651 23969 i
26  * 203.119.104.1 0 4608 24115 6939 4651 23969 i
27  * 203.119.104.2 0 4608 24115 6939 4651 23969

```

Consequently, we can NOT derive a rule to estimate the compression performance based on the number of next-hop addresses present in the table. We can’t predict how prefixes are linked to each NH and how they are distributed. It shows the limits of what we can do in the lab. To estimate the compression benefits before deploying the PTX in production, you’ll need the full output of routes and next-hop information. The real life numbers presented in the chart above are the most definitive proof of the algorithm efficiency.

## 1.7 Best Case Scenario

To understand how far the current internet table can be compressed, we advertised the internet routes present in <https://bgp.potaroo.net/as2.0/bgptable.txt> to “single-attached” router.



**Figure 12:** Best Case Test Topology

Of course, a single default route would do the same job ;)

But the purpose of this test is to identify how far we can compress a current internet table if all existing routes point to the same next hop. That represents the best case we can reach with this implementation.

```

1 regress@rtme-ptx10> show bgp summary
2
3 Threading mode: BGP I/O
4 Default eBGP mode: advertise - accept, receive - accept
5 Groups: 4 Peers: 6 Down peers: 4
6 Table Tot Paths Act Paths Suppressed History Damp State Pending
7 inet.0
8           930416 930416 0 0 0 0
9 inet6.0
10          161443 161443 0 0 0 0
11 Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State|#Active/Received/Accepted/Damped...
12 15.1.1.2 65001 93 34 0 5 7 Active
13 15.1.2.2 65002 0 0 0 4 10:39 Active
14 15.1.3.2 65003 1492 73 0 4 23 Establ
15   inet.0: 930416/930416/930416/0
16 2002:15:1:1::2 65001 0 0 0 2 9:50 Active
17 2002:15:1:2::2 65002 0 0 0 2 10:38 Active
18 2002:15:1:3::2 65003 345 7 0 2 2:46 Establ
19   inet6.0: 161443/161443/161443/0
20
21 regress@rtme-ptx10>

```

We advertise 930,416 IPv4 and 161,433 IPv6 prefixes with the same next-hop address. And we check the compression at the PFE level:

```

1 regress@rtme-ptx10:pfe> show route compression
2
3 Index Proto Prefixes Aggregate Installed Comp(%)
4 -----
5 0 IPv4 911507 517408 167198 82
6 1 IPv4 0 0 0 0
7 51 IPv4 5 0 5 0
8 36738 IPv4 9 0 9 0
9 0 IPv6 159527 68148 46260 72
10 1 IPv6 0 0 0 0
11 51 IPv6 6 0 6 0
12 36738 IPv6 6 0 5 17
13
14 regress@rtme-ptx10:pfe>

```

It's an interesting finding. In September 2022, with the internet view proposed by potaroo.net, we can compress the IPv4 table by 82% (that means it will occupy only 18% of the space it would have used without compression) and the IPv6 table by 72%.

Again, it's an best-case scenario for internet table. But your table can potentially contain many IGP routes that can be compressed too.

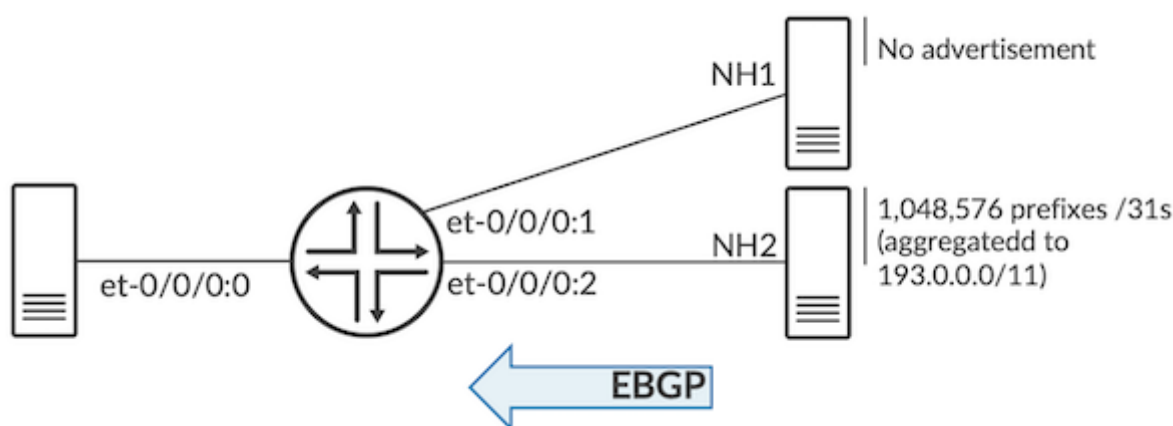
## 1.8 What About the Churn?

What happens when a network event triggers the rebuild of the radix tree and the re-installation of FIB table blocks? It's a legitimate question since the network and internet are not static, you may receive new routes, or existing routes could be resolved by a new Next-Hop address (a different peering point for example).

Like every Junos process, the FIB compression implementation follows a make-before-break logic. That means that all the changes are brought into the FIB before we remove the previous entries. It guarantees we don't create any black holes while the system is converging.

We will run the following test in the lab to demonstrate the compression algorithm doesn't cause any packet drop while re-constructing a large tree.

Let's start with a very big aggregation of 1M contiguous /31 routes into a single /11.



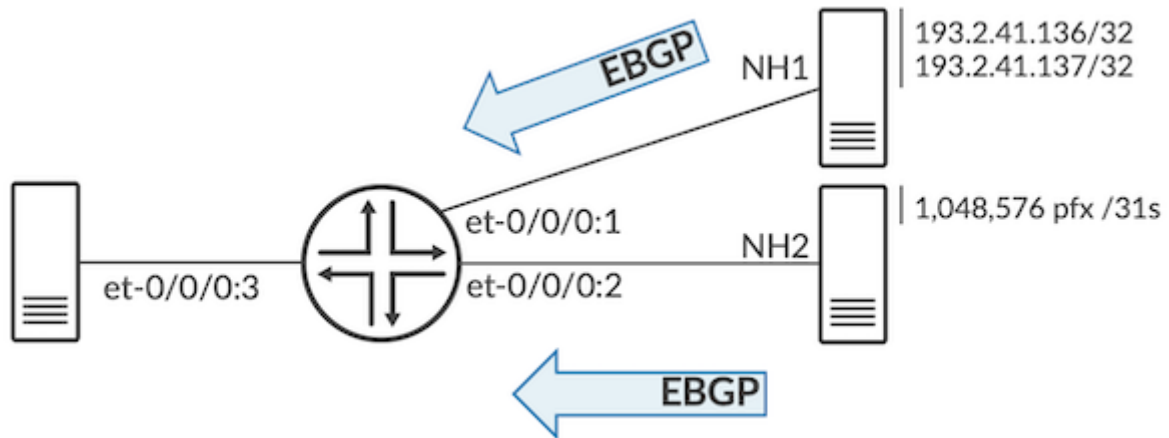
**Figure 13:** Advertisement of 1M Contiguous Prefixes

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select installed
2
3
4 Index Destination NH Id NH Type NH Token GUID
5 -----
6 0 default 34 discard 1140 833223655665
7 0 0.0.0.0 34 discard 1140 622770257990
8 <SNIP>
9 0 15.1.3.255 11034 bcast 5464 841813591844
10 0 193.0/11 13036 software 6104 0
11 0 224/4 35 mdiscard 1141 622770257992
12 0 224.0.0.1 31 mcast 1137 622770257985
13 0 255.255.255.255 32 bcast 1138 622770257987
14
15 regress@rtme-ptx10:pfe>

```

Now, we break the aggregation structure with the advertisement of two /32s in the middle of this perfect alignment via a different eBGP peer (therefore, a different NH address)



**Figure 14:** Additional Advertisement of Two /32 Prefixes

```

1 regress@rtme-ptx10:pfe> show route proto ip index 0 select installed
2
3
4 Index Destination NH Id NH Type NH Token GUID
5 -----
6 0 default 34 discard 1140 833223655665
7 0 0.0.0.0 34 discard 1140 622770257990
8 0 12.1.1.1 11002 local 1308 841813590953
9 <SNIP>
10 0 15.1.3.255 11034 bcast 5464 841813591844
11 0 193.0/15 13036 software 6104 0
12 0 193.2.0/19 13036 software 6104 0
13 0 193.2.32/21 13036 software 6104 0
14 0 193.2.40/24 13036 software 6104 0
15 0 193.2.41.0/25 13036 software 6104 0
16 0 193.2.41.128/29 13036 software 6104 0
17 0 193.2.41.136/31 13037 software 6106 0
18 0 193.2.41.138/31 13036 software 6104 833231556171
19 0 193.2.41.140/30 13036 software 6104 0
20 0 193.2.41.144/28 13036 software 6104 0
21 0 193.2.41.160/27 13036 software 6104 0
22 0 193.2.41.192/26 13036 software 6104 0
23 0 193.2.42/23 13036 software 6104 0
24 0 193.2.44/22 13036 software 6104 0
25 0 193.2.48/20 13036 software 6104 0
26 0 193.2.64/18 13036 software 6104 0
27 0 193.2.128/17 13036 software 6104 0
28 0 193.3/16 13036 software 6104 0
29 0 193.4/14 13036 software 6104 0
30 0 193.8/13 13036 software 6104 0
31 0 193.16/12 13036 software 6104 0
32 0 224/4 35 mdiscard 1141 622770257992
33 0 224.0.0.1 31 mcast 1137 622770257985
34 0 255.255.255.255 32 bcast 1138 622770257987
35
36 regress@rtme-ptx10:pfe>

```

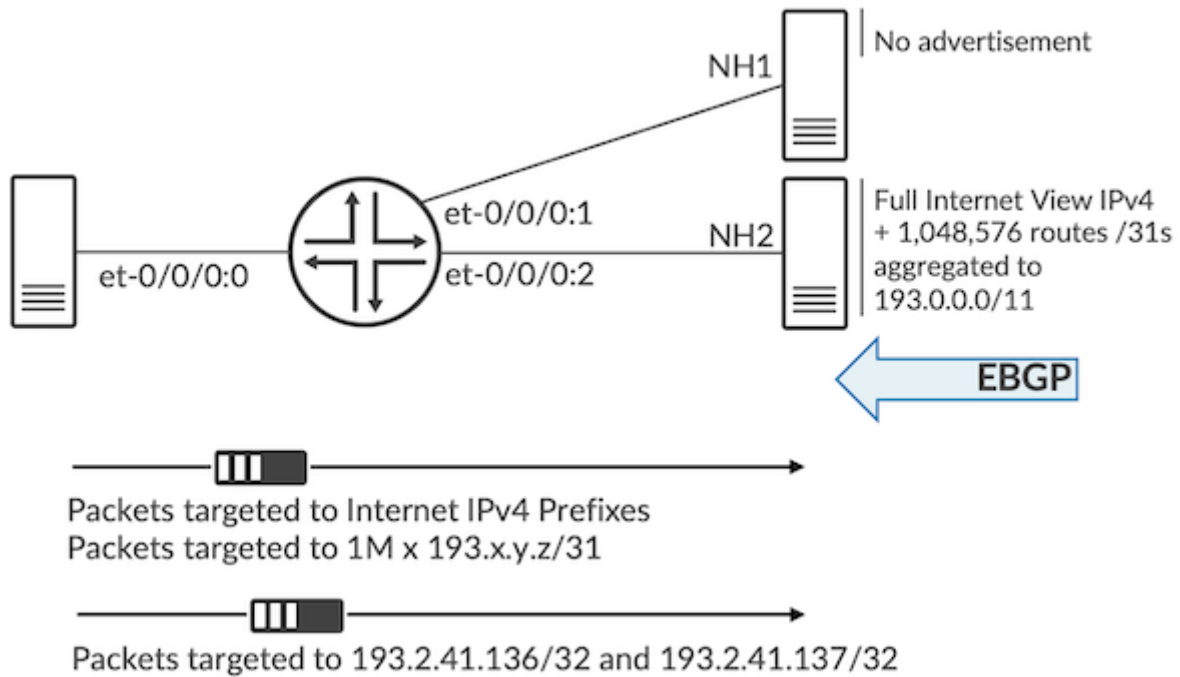
The introduction of these two routes reshuffled the compression and we have 21 entries programmed in the FIB instead of one.

Now that we know what the advertisement of these two prefixes does on the compression structure, let's verify the potential collateral impact on traffic.

We will move back and forth between two “states” in the lab.

### State 1:

A full internet v4 table is advertised on top of the previous million /31s entries. And we generate traffic to these prefixes. All of them. It represents more or less 2M routes, and streams.



**Figure 15:** Churn Test – State 1

```

1 regress@rtme-ptx10> show route 193.2.41.136
2
3
4 inet.0: 1979001 destinations, 1979001 routes (1979001 active, 0 holddown, 0 hidden)
5 + = Active Route, - = Last Active, * = Both
6
7 193.2.41.136/31 *[BGP/170] 12:59:11, localpref 100
8     AS path: 65003 I, validation-state: unverified
9     > to 15.1.3.2 via et-0/0/0:2.0
10
11 mgmt_junos.inet.0: 3 destinations, 3 routes (3 active, 0 holddown, 0 hidden)
12 + = Active Route, - = Last Active, * = Both
13
14 0.0.0.0/0 *[Static/5] 3d 16:32:15
15     > to 10.83.153.254 via re0:mgmt-0.0
16
17 regress@rtme-ptx10> show route 193.2.41.137
18
19 inet.0: 1979001 destinations, 1979001 routes (1979001 active, 0 holddown, 0 hidden)
20 + = Active Route, - = Last Active, * = Both
21
22 193.2.41.136/31 *[BGP/170] 12:59:15, localpref 100
23     AS path: 65003 I, validation-state: unverified
24     > to 15.1.3.2 via et-0/0/0:2.0
25
26 mgmt_junos.inet.0: 3 destinations, 3 routes (3 active, 0 holddown, 0 hidden)
27 + = Active Route, - = Last Active, * = Both
28
29 0.0.0.0/0 *[Static/5] 3d 16:32:19
30     > to 10.83.153.254 via re0:mgmt-0.0
31
32 regress@rtme-ptx10>

```

With the internet routes and all the million contiguous /31 prefixes, the compression ratio reaches extremely high levels:

```

1 regress@rtme-ptx10:pfe> show route summary
2
3
4 IPv4 Route Tables:
5 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
6 -----
7 Default 1995906 369716672 1964061 1559062 163226 92
8 1 0 0 0 0 0

```



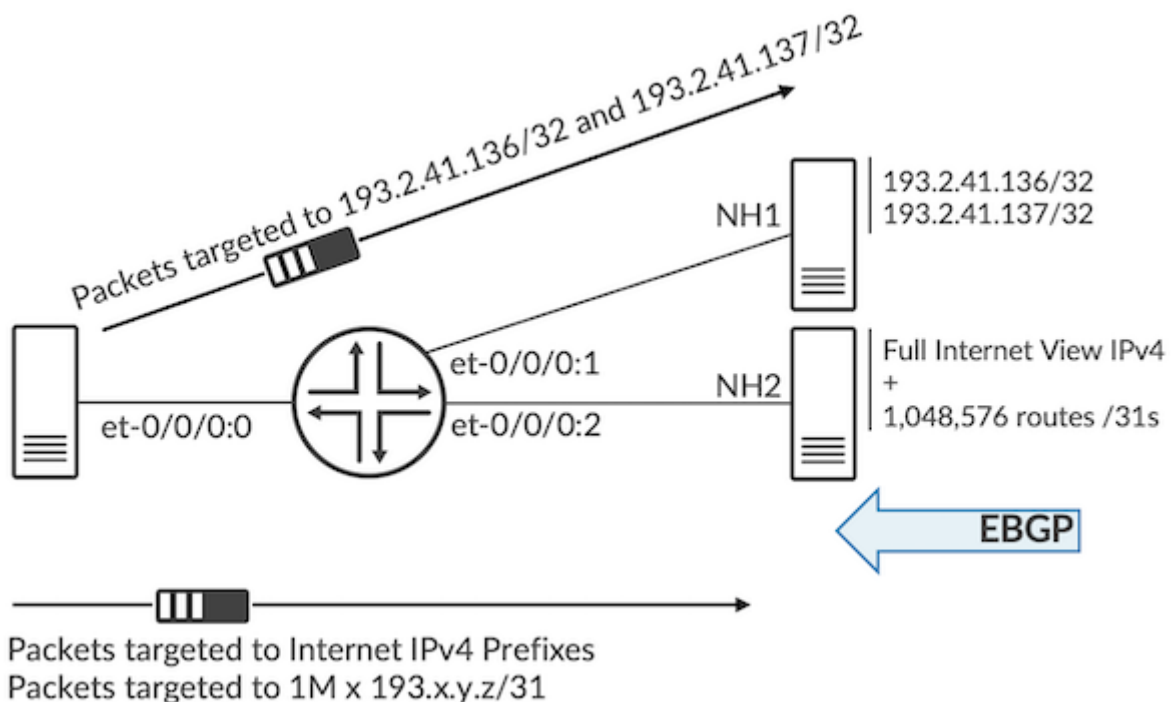
```

9  51 5 520 5 0 5 0
10 36738 9 936 9 0 9 0
11
12 MPLS Route Tables:
13 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
14 -----
15 Default 1 104 1 0 1 -
16 52 1 104 1 0 1 -
17
18 IPv6 Route Tables:
19 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
20 -----
21 Default 323 33592 29 0 27 7
22 1 0 0 0 0 0 0
23 51 6 624 6 0 6 0
24 36738 7 728 6 0 5 17
25
26 CLNP Route Tables:
27 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
28 -----
29 Default 1 104 1 0 1 -
30 51 1 104 1 0 1 -
31
32 regress@rtme-ptx10:pfe>

```

## State 2:

We advertise the two prefixes from a different next-hop, breaking the 1M aggregation and creating a re-computation of the tree, while having the “background traffic” of all internet routes.



**Figure 16:** Churn Test – State 2

```

1  regress@rtme-ptx10> show route 193.2.41.136
2
3  inet.0: 1979003 destinations, 1979003 routes (1979003 active, 0 holddown, 0 hidden)
4  + = Active Route, - = Last Active, * = Both
5
6  193.2.41.136/32 *[BGP/170] 00:00:12, localpref 100
7                    AS path: 65002 I, validation-state: unverified
8                    > to 15.1.2.2 via et-0/0/0:1.0
9
10 mgmt_junos.inet.0: 3 destinations, 3 routes (3 active, 0 holddown, 0 hidden)
11 + = Active Route, - = Last Active, * = Both

```

```

12
13 0.0.0.0/0 *[Static/5] 3d 16:33:58
14   > to 10.83.153.254 via re0:mgmt-0.0
15
16 regress@rtme-ptx10> show route 193.2.41.137
17
18 inet.0: 1979003 destinations, 1979003 routes (1979003 active, 0 holddown, 0 hidden)
19 + = Active Route, - = Last Active, * = Both
20
21 193.2.41.137/32 *[BGP/170] 00:00:09, localpref 100
22   AS path: 65002 I, validation-state: unverified
23   > to 15.1.2.2 via et-0/0/0:1.0
24
25 mgmt_junos.inet.0: 3 destinations, 3 routes (3 active, 0 holddown, 0 hidden)
26 + = Active Route, - = Last Active, * = Both
27
28 0.0.0.0/0 *[Static/5] 3d 16:33:55
29   > to 10.83.153.254 via re0:mgmt-0.0
30
31 regress@rtme-ptx10>

```

We have this background traffic going to every internet prefix in the table and every one of these 1M /31 prefixes. Plus, we have this specific stream block for the two /32 prefixes that will be received via et-0/0/0:2 or et-0/0/0:3 depending on the advertisement.

On the traffic/route generator, we will alternate advertisements and withdrawals.

After 10 changes, we check the total number of packets on both ports (verifying we received as much as we sent).

| Port Name     | Port Count (Frames) | Generator Sig Count (Frames) | Rx Sig Count (Frames) | Total Tx Rate (fps) | Total Rx Rate (fps) | Generator Rate (fps) | Gene |
|---------------|---------------------|------------------------------|-----------------------|---------------------|---------------------|----------------------|------|
| Port //5/9... | 77,114              | 695,477,114                  | 0                     | 0                   | 0                   | 0                    | 0    |
| Port //5/1... | 0                   | 0                            | 460,108               | 0                   | 0                   | 0                    | 0    |
| Port //5/1... | 0                   | 0                            | 695,017,006           | 0                   | 0                   | 0                    | 0    |
| Σ             | 77,114              | 695,477,114                  | 695,477,114           |                     |                     |                      |      |

**Figure 17:** IXIA Results

695,477,144 packets sent and received: As expected, not a single packet dropped in this experiment.

We understand that we can't go very far in a lab, but it demonstrates the make-before-break approach used in our implementation. No impact on the prefixes being compressed or "de-aggregated" and no impact on the traffic carried by other prefixes in the table.

## 1.9 Useful links

- <https://bgp.potaroo.net/as2.0/bgptable.txt>

- <https://www.cidr-report.org/as2.0/>

## 1.10 Glossary

- AFT: Advanced Forwarding Toolkit
- AFTman: AFT Manager
- CDA: Common Driver ASIC driver
- DDS: Distributed DataStore
- FIB: Forwarding Information Base
- fibd: FIB daemon
- LC: Line Card
- NH: Next-Hop (address)
- OFP: Object Flooding Protocol
- RE: Routing Engine
- rpd: route processor daemon
- WR: WindRiver Linux

## 1.11 Acknowledgements

Many thanks to Suneesh Babu, Dmitry Shokarev, Dmitry Bugrimenko, Edward Ricioppo, Zuhair Makawa, Kevin F Wang and Alex Varghese for their help describing the FIB compression concepts, testing it in our Sunnyvale labs, and collecting data from customer deployments.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

**HEWLETT PACKARD ENTERPRISE**

**HPE.com**

**HPE**

